

From Heuristics to Generative Agents: A Paradigm Shift in Automated Program Repair Capabilities and Robustness

Automated Program Repair (APR) has undergone a fundamental transformation, evolving from rigid, search-based heuristics constrained by the "plastic surgery hypothesis" to flexible, generative approaches powered by Large Language Models (LLMs) [162] [68]. While traditional methods relied on predefined templates and limited search spaces, modern LLMs synthesize novel patches by leveraging vast pre-training corpora, effectively transcending earlier redundancy assumptions. However, this paradigm shift introduces critical challenges regarding efficiency, robustness, and evaluation validity. To address inference latency, recent innovations include Non-Autoregressive models like NARRepair and Parameter-Efficient Fine-Tuning strategies that balance computational cost with repair accuracy [10] [277]. Simultaneously, the field grapples with model fragility, where syntactic variations and data leakage threaten generalizability, necessitating robust benchmarks like HEJ-Robust [112] [84]. The current trajectory emphasizes agentic frameworks and Retrieval-Augmented Generation, which integrate dynamic execution feedback and repository-level context to mimic human debugging workflows [32] [27]. This synthesis highlights a maturing field moving beyond mere patch generation toward trustworthy, context-aware, and efficient autonomous software maintenance systems.

Introduction to the Paradigm Shift in Automated Program Repair

The landscape of Automated Program Repair (APR) has undergone a fundamental transformation, evolving from rigid, search-based heuristics to flexible, data-driven approaches powered by Large Language Models (LLMs). Historically, the field was dominated by generate-and-validate paradigms that relied heavily on the "plastic surgery hypothesis," which posits that correct code ingredients for a fix already exist elsewhere within the same project [162]. Traditional techniques, including search-based methods like GenProg and template-based approaches such as TBar, navigated explicitly defined search spaces or applied predefined fix patterns to generate patches [55]. While these methods offered interpretability, they were inherently limited by the scope of their manually crafted rules and the redundancy assumption, often failing to synthesize novel fixes for complex or unseen bug types [180]. Furthermore, these early approaches frequently struggled with patch overfitting, where generated patches passed the provided test suites but failed to generalize to unseen behaviors, a persistent challenge that undermined developer trust in automated solutions [162].

The advent of deep learning marked an initial shift toward learning-based APR, framing repair as a Neural Machine Translation (NMT) task where buggy code is translated into fixed code [23]. However, the true paradigm shift occurred with the integration of Large Language Models, which leverage vast pre-training corpora to understand complex semantic contexts and synthesize novel patches without explicit rule definitions [68]. Unlike their predecessors, modern Code LLMs do not merely copy and paste existing code; they possess the generative capacity to create entirely new logic, effectively transcending the limitations of the plastic surgery hypothesis [199]. Research indicates that directly applying state-of-the-art pre-trained language models can substantially outperform all existing traditional and learning-based APR techniques across diverse datasets, demonstrating a scaling effect where larger models tend to achieve superior repair performance [6]. This capability is further enhanced by the ability of LLMs to generate high-quality synthetic training data, addressing the scarcity of diverse bug-fix pairs and enabling models to learn from a broader spectrum of software defects [180].

This evolution has also redefined the repair workflow itself, moving away from the strict dependency on precise, statement-level fault localization. While traditional methods required costly fine-grained localization to constrain the search space, LLM-based approaches have enabled function-level and even repository-level repair [96]. Innovations such as RepoRepair leverage LLMs to generate hierarchical code documentation, allowing models to comprehend repository-level context and dependencies that were previously inaccessible to automated tools [27]. Moreover, the rigidity of static analysis has been replaced by dynamic, iterative reasoning. Approaches like DynaFix and DebugRepair integrate execution-level dynamic information and self-directed debugging into the repair loop, enabling models to simulate the stepwise debugging practices of human developers by analyzing variable states and control-flow paths rather than relying solely on coarse pass/fail test results [119; 32]. This shift allows for a more nuanced understanding of root causes, particularly in complex multi-hunk bugs where static context is insufficient.

Despite these advancements, the transition to LLM-driven APR introduces new complexities regarding model adaptation and efficiency. The community is currently navigating a tension between full-model fine-tuning and parameter-efficient strategies. While some studies suggest that full fine-tuning can lead to overfitting on specific data distributions, parameter-efficient fine-tuning (PEFT) methods like LoRA and IA3 have shown promise in adapting large models to repair tasks with lower computational costs and better generalization [132; 277]. Additionally, the efficiency of patch generation remains a critical concern; techniques like FLAMES have been developed to address the memory inefficiency of beam search in large models by employing semantic-guided search strategies, significantly reducing VRAM consumption while maintaining high repair rates [240]. The issue of overfitting also persists in a new form, prompting the development of sophisticated patch correctness assessment tools like ComPass, which utilize contrastive learning to distinguish between semantically correct fixes and those that merely satisfy test constraints [205].

The current research trajectory emphasizes not just the generation of patches, but the alignment of LLM objectives with repair tasks and the optimization of input representations. Studies suggest that aligning the output objective with the model's pre-training goal, such as using cloze-style tasks instead of direct translation, can significantly enhance repair capabilities [44; 20]. Simultaneously, there is a growing recognition that providing LLMs with richer context—such as code reviews, issue descriptions, and historical fix patterns—through retrieval-augmented generation can further boost

performance [31; 1]. As the field matures, the focus is shifting from merely demonstrating feasibility on standard benchmarks like Defects4J to ensuring robustness, generalizability across programming languages, and practical applicability in real-world industrial settings [62; 84]. This paradigm shift represents a move from heuristic-driven, localized fixes to a holistic, context-aware, and generative approach that fundamentally alters how software maintenance is conceptualized and executed.

Evolution from Heuristic to Generative Approaches

The trajectory of Automated Program Repair (APR) has undergone a fundamental paradigm shift, evolving from rigid, rule-based systems to flexible, generative models capable of understanding and synthesizing code with human-like proficiency. Early APR techniques were predominantly heuristic-based, constraint-based, or template-based, relying heavily on manually crafted rules and predefined patterns to navigate the search space of potential fixes. As noted in [55], these traditional approaches, while effective for specific, well-understood bug types, struggled significantly with the diversity and complexity of real-world software defects. Template-based methods, often considered state-of-the-art among traditional techniques, required immense expert effort to design repair templates for specific bug categories, such as null pointer exceptions, which inherently limited their applicability to unseen or novel bug patterns [23]. Furthermore, search-based approaches faced insurmountable challenges due to vast search spaces, often failing to locate correct solutions within reasonable timeframes, while constraint-based methods were severely limited by the availability of precise formal specifications [28]. These limitations highlighted a critical bottleneck: the inability of static, manually engineered rules to generalize across the dynamic landscape of modern software development.

The advent of Deep Learning (DL) marked the first major transition, introducing Neural Machine Translation (NMT) paradigms to the field. This approach reframed APR as a translation task, where buggy code snippets serve as the source language and correct code snippets as the target language [23]. Techniques such as SequenceR and CURE utilized encoder-decoder architectures, typically based on Recurrent Neural Networks (RNNs) or early Transformers, to learn bug-fixing patterns from massive corpora of open-source code [53]. While these learning-based techniques achieved remarkable performance compared to their heuristic predecessors by automatically learning hidden relationships in data, they still faced significant limitations. Specifically, NMT-based approaches often generated syntactically incorrect patches or struggled with long-range dependencies in code, and their effectiveness was heavily contingent on the quality and size of the training datasets [181]. Additionally, the autoregressive nature of these models resulted in significant inference delays, making real-time repair impractical for complex bugs [10]. To address these latency issues, researchers developed non-autoregressive (NAR) models like NARRepair, which generate target code in parallel, significantly improving inference speed while attempting to maintain patch quality through action-guided repair and AST-based dependency extraction [188]. However, even with these architectural optimizations, the fundamental limitation of treating repair purely as a sequence-to-sequence translation task remained, as these models often lacked the broader contextual understanding and reasoning capabilities necessary for complex, multi-hunk bugs.

The most recent and transformative evolution in APR is the integration of Large Language Models (LLMs), which has superseded earlier DL approaches by enabling multi-step reasoning, few-shot learning, and zero-shot capabilities across diverse programming languages. Unlike previous models that required extensive fine-tuning on specific bug-fix datasets, modern LLMs leverage vast pre-training on general code and natural language, allowing them to generalize to unseen bug types and languages with minimal adaptation [6]. Research indicates that LLMs can outperform state-of-the-art deep learning techniques even without fine-tuning, utilizing their latent knowledge to predict correct code based on context rather than merely translating buggy sequences [44]. This shift has enabled novel repair strategies, such as "cloze-style" repair where models fill in masked code segments, and conversational repair frameworks that iteratively refine patches based on test feedback, as demonstrated by tools like ChatRepair and ContrastRepair [85; 201].

Furthermore, the generative capacity of LLMs has facilitated the move beyond single-line fixes to function-level and even repository-level repair, addressing multi-hunk bugs that were previously intractable for template-based or standard NMT systems [96]. The flexibility of LLMs also allows for the incorporation of diverse contextual information, including error messages, test cases, and historical commit data, to guide the repair process more effectively [65]. However, this evolution is not without challenges; issues such as hallucination, overfitting to training data, and the high computational cost of inference remain active areas of investigation [112]. Recent studies also highlight the importance of aligning the model's objective with the repair task, suggesting that direct debugging workflows may be more effective than traditional localize-then-repair pipelines for LLMs [20]. The field continues to evolve with hybrid approaches that combine the reasoning power of LLMs with formal program analysis and retrieval-augmented generation to ensure semantic correctness and reduce the search space, representing the current frontier in automated software maintenance [51; 15]. Additionally, emerging paradigms like specification-centric repair, which focuses on

correcting behavior specifications before generating code, illustrate the move away from purely code-centric modifications toward a deeper understanding of program intent [90].

Challenges in Trust, Robustness, and Evaluation

The integration of Large Language Models (LLMs) into Automated Program Repair (APR) represents a paradigm shift from rigid, template-based heuristics to generative, context-aware solutions, yet this transition introduces profound challenges regarding trust, robustness, and evaluation validity. A primary erosion of trust stems from the propensity of LLMs to hallucinate patches that are syntactically valid but semantically flawed, often propagating vulnerabilities inherent in their training corpora. Research indicates that models trained on public repositories, such as Codex, can generate verbatim "simple, stupid bugs" at twice the rate of correct code, raising concerns about the safety of deploying such models in security-critical domains without rigorous human oversight [146]. This lack of trust is compounded by the opacity of black-box models, where developers cannot easily verify the reasoning behind a generated fix, particularly when LLMs hallucinate API parameters or select incorrect libraries, necessitating expensive manual review processes [256] [183]. Furthermore, while conversational agents like ChatRepair demonstrate the potential to fix bugs at low monetary costs by leveraging feedback loops, the reliance on iterative refinement can lead to prohibitive token consumption for complex repairs, creating a tension between cost-efficiency and repair effectiveness [85] [164].

Robustness remains a critical bottleneck, as LLM-based APR systems exhibit extreme sensitivity to syntactic variations and prompt formulations that should theoretically be semantically equivalent. Empirical studies utilizing metamorphic testing reveal that semantically equivalent transformations, such as variable renaming or statement reordering, cause state-of-the-art repair tools to fail in a significant portion of cases, exposing a fragility in their reasoning capabilities that contradicts the expectation of semantic understanding [127]. This instability is exacerbated by the "lost-in-the-middle" phenomenon, where models fail to retain key failure-inducing information when prompts contain extensive test inputs, leading to degraded performance unless inputs are aggressively reduced [47]. Moreover, the effectiveness of these models is heavily contingent on the quality of fault localization; under realistic assumptions of imperfect fault localization, repair accuracy drops significantly compared to evaluations that assume perfect bug location, creating a misleading gap between academic benchmarks and industrial reality [62] [96]. The sensitivity also extends to the model's training objective, where a misalignment between next-token prediction and the masked span prediction required for infilling can impede the model's ability to leverage pre-trained knowledge effectively [20].

Evaluation methodologies for LLM-based APR are currently fraught with contradictions, primarily due to an over-reliance on the Defects4J dataset, which raises severe concerns about data leakage and dataset overfitting given that many LLMs were trained on code overlapping with these benchmarks [112]. Studies utilizing transformed datasets, such as DEFECTS4J-TRANS, reveal that the average number of correct and plausible patches decreases by nearly 50% when models are tested on semantically equivalent but syntactically varied code, indicating that current models lack true generalizability [112]. Traditional metrics further fail to capture patch quality, as a patch may pass all available test cases (plausible) yet be incorrect or overfitted, a problem that standard test-suite-based validation cannot fully address without advanced assessment techniques like contrastive learning [205]. Additionally, many existing robustness tests employ unnatural code transformations that do not reflect real-world coding practices, potentially misleading researchers about the true resilience of APR tools and highlighting the need for evaluation frameworks that account for natural code variations [299]. The diversity of programming languages also poses a challenge, as models that excel in Java may perform poorly in C/C++ or Python, suggesting that a single model cannot universally address repair tasks across different ecosystems without specialized benchmarks like Defects4C [257]. Consequently, the field lacks a unified, realistic evaluation framework that accounts for imperfect localization, diverse languages, and natural code variations, hindering the reliable assessment of trust and robustness in LLM-based APR systems. Future research must prioritize the development of specification-centric repair mechanisms that integrate formal verification to mitigate hallucination, as well as federated learning approaches to leverage private industrial data without compromising privacy, thereby bridging the gap between academic prototypes and deployable industrial solutions [284] [220].

Model Architectures, Adaptation, and Efficiency Strategies

The integration of Large Language Models (LLMs) into Automated Program Repair (APR) has fundamentally shifted the research paradigm from heuristic search to generative adaptation, yet the optimal strategy for adapting these massive models remains a subject of intense debate. While pre-trained models possess inherent coding knowledge, empirical evidence suggests that direct application without task-specific adaptation often yields suboptimal results compared to tailored approaches [60]. The central tension in current literature lies in balancing model scale, specialization, and computational cost. Early investigations revealed that simply applying pre-trained language models (PLMs) directly to APR tasks can outperform traditional techniques, yet significant gains are achieved when these models are adapted through fine-tuning [73]. However, the efficacy of Full-Model Fine-Tuning (FMFT) versus Parameter-Efficient Fine-Tuning (PEFT) presents a critical divergence in findings. On one hand, FMFT was initially the standard, but recent studies indicate it can lead to overfitting and decreased performance on benchmark datasets due to distributional shifts between pre-training and repair data [132]. Conversely, other research utilizing instruction-tuned models challenges this notion, suggesting that even using a fraction of a fine-tuning dataset can yield improvements of up to 78% in plausible patch generation, contradicting reports of limited gains from full fine-tuning [241]. This discrepancy highlights that the success of FMFT is highly dependent on dataset quality and the specific training regimen employed.

In response to the risks of overfitting and the high computational burden of FMFT, Parameter-Efficient Fine-Tuning (PEFT) has emerged as a robust alternative. Methods such as Low-Rank Adaptation (LoRA) and IA3 restrict the number of trainable parameters, often resulting in better generalization and higher fixing capabilities. Research demonstrates that IA3 can enhance the creativity of LLMs more effectively than other PEFT methods, achieving state-of-the-art results while significantly reducing memory usage and computational overhead [277]. This efficiency is further corroborated by findings that PEFT allows models to fix significantly more bugs than state-of-the-art LLM-based techniques without the resource burden of full fine-tuning [54]. Beyond parameter updates, the objective function of fine-tuning plays a pivotal role. Standard fine-tuning often focuses solely on syntactic code transformation, potentially neglecting the logical reasoning behind repairs. To address this, multi-objective fine-tuning strategies have been proposed, such as MORepair, which jointly optimizes for code transformation and the logical reasoning behind changes using natural language explanations. This approach has been shown to boost repair performance significantly across various model sizes and architectures, narrowing the gap between small open-source models and larger closed-source counterparts [67]. Similarly, preservation-aware fine-tuning methods like PAFT aim to minimize edit distance by explicitly signaling which tokens should be preserved, thereby reducing over-editing and producing more localized, plausible patches [35].

Architectural innovations also extend to the decoding process itself to address latency issues inherent in traditional autoregressive (AR) models. Non-autoregressive (NAR) models, such as NARRepair, have been introduced to parallelize code generation, offering speedups of up to 18 times while maintaining competitive accuracy through novel mechanisms like action-guided repair and inter-token dependency extraction [188] [10]. Additionally, search strategies during inference are being re-evaluated; while increasing beam size typically improves performance, it often leads to memory overloads. Novel approaches like FLAMES employ semantic-guided search to navigate the patch space efficiently without the heavy memory costs associated with large beam sizes [240]. Cost-efficient strategies like CIGAR further optimize prompt settings to minimize token usage while maintaining repair effectiveness [164].

The role of data scarcity and privacy further complicates adaptation strategies. In scenarios with limited data, prompt tuning augmented with domain knowledge has been shown to outperform fine-tuning, offering a viable alternative when large datasets are unavailable [226]. Moreover, federated learning has emerged as a promising avenue for fine-tuning LLMs on private, decentralized industrial data, enabling collaborative repair capabilities without compromising data privacy [284]. Finally, the generation of synthetic data to augment training sets has demonstrated potential in overcoming data limitations, with quality-filtered synthetic datasets outperforming real-world commit data in certain repair scenarios [180]. These diverse strategies collectively underscore that the effectiveness of LLMs in APR is not merely a function of model scale but is heavily dependent on the nuanced interplay between architectural design, adaptation methodology, and efficiency optimizations.

Fine-Tuning Paradigms and Parameter Efficiency

The adaptation of Large Language Models (LLMs) for Automated Program Repair (APR) has undergone a critical paradigm shift, moving away from the assumption that Full-Model Fine-Tuning (FMFT) is the optimal strategy for task specialization. While FMFT was initially the standard for aligning pre-trained models with repair tasks, empirical evidence now suggests it often incurs prohibitive computational costs and introduces significant risks of catastrophic forgetting and overfitting, particularly when training data distributions diverge from inference scenarios. A primary manifestation of this misalignment is the "over-editing" phenomenon, where models generate patches that are functionally correct but involve excessive rewriting of stable code, thereby increasing the cognitive load on developers during code review. To address these dual challenges of edit precision and resource efficiency, recent research has prioritized the development of Preservation-Aware Fine-Tuning (PAFT) and Parameter-Efficient Fine-Tuning (PEFT) strategies. These approaches aim to minimize unnecessary code modifications and reduce trainable parameters while maintaining or enhancing repair accuracy, effectively decoupling repair correctness from the magnitude of code changes.

Standard supervised fine-tuning often fails to provide explicit signals regarding which tokens should remain unchanged, leading models to treat the entire code sequence as mutable. PAFT: Preservation-Aware Fine-Tuning for Minimal-Edit Program Repair addresses this limitation by introducing a methodology that explicitly aligns buggy and fixed code to derive token-level preservation signals. By combining these signals with full-sequence masking and an edit-difficulty curriculum, PAFT forces the model to concentrate edits strictly on faulty regions. Empirical results demonstrate that this approach significantly reduces Average Edit Distance (AED) by up to 32.6% while improving pass@1 rates, proving that preserving stable context is as crucial as generating correct fixes. Similarly, QiMeng-PRepair: Precise Code Repair via Edit-Aware Reward Optimization tackles over-editing through reinforcement learning, utilizing an edit-aware reward function within a Group Relative Policy Optimization framework. This method encourages minimal yet correct edits, enhancing repair precision without requiring expensive inference-time search or reranking, thus offering a distinct advantage over standard fine-tuning objectives that prioritize syntactic transformation over edit minimality.

Parallel to preservation awareness, the field has extensively explored PEFT to mitigate the computational burden and instability associated with updating all model parameters. Contrary to the intuition that more trainable parameters yield better performance, The Impact of Fine-tuning Large Language Models on Automated Program Repair observes that FMFT can sometimes degrade benchmark performance due to data distribution shifts and overfitting. In contrast, restricting the scope of weight updates via PEFT methods such as LoRA and $(IA)^3$ often provides a more stable adaptation mechanism. This finding is corroborated by Exploring Parameter-Efficient Fine-Tuning of Large Language Model on Automated Program Repair, which demonstrates that $(IA)^3$ can achieve superior bug-fixing capabilities compared to state-of-the-art FMFT approaches while consuming fewer resources. Notably, this study highlights that increasing the number of trainable parameters or the size of the instruction dataset does not linearly correlate with performance gains in PEFT settings, suggesting an optimal configuration exists that balances efficiency and effectiveness. Furthermore, The Art of Repair: Optimizing Iterative Program Repair with Instruction-Tuned Models reveals that even with limited fine-tuning data (less than 1% of typical datasets), instruction-tuned models can achieve substantial improvements, though exceeding certain thresholds leads to diminishing returns, reinforcing the need for parameter-efficient constraints.

Building on these efficiency gains, RepairLLaMA: Efficient Representations and Fine-Tuned Adapters for Program Repair pioneers the use of fine-tuned adapters combined with optimized code representations. RepairLLaMA demonstrates that parameter-efficient strategies not only facilitate faster convergence but also enhance the model's ability to generalize to bugs outside the fine-tuning distribution, outperforming baselines that rely on naive code representations and full-model updates. The efficacy of fine-tuning is further nuanced by the objectives employed during training. MOREpair: Teaching LLMs to Repair Code via Multi-Objective Fine-tuning argues that standard fine-tuning often overlooks the logical reasoning behind code changes. By introducing a multi-objective framework that jointly optimizes for code transformation and natural language explanations of the repair logic, MOREpair enables models to internalize the "why" behind fixes, yielding significant performance improvements across different model sizes. Additionally, in data-scarce scenarios where fine-tuning is infeasible, Adapting Knowledge Prompt Tuning for

Enhanced Automated Program Repair suggests that prompt tuning augmented with domain knowledge can outperform traditional fine-tuning, offering a viable alternative when large labeled datasets are unavailable. Collectively, these studies illustrate that the future of efficient APR lies not merely in reducing parameter counts, but in strategically aligning fine-tuning objectives with preservation constraints, logical reasoning, and robustness requirements to produce minimal, high-quality repairs.

Prompt Engineering and Zero-Shot Capabilities

The paradigm of Automated Program Repair (APR) has undergone a fundamental shift with the advent of Large Language Models (LLMs), challenging the long-held assumption that extensive task-specific fine-tuning is a prerequisite for effective repair. Contrary to traditional learning-based approaches that frame APR as a Neural Machine Translation (NMT) problem requiring large datasets of buggy-fixed pairs, recent research demonstrates that pre-trained code models possess inherent repair capabilities that can be unlocked through zero-shot prompting and cloze-style prediction tasks. This shift suggests that the statistical knowledge of code naturalness and syntax acquired during pre-training is often sufficient to identify and correct defects without additional gradient updates on repair-specific data [44]. A pivotal development in this domain is the reframing of program repair as a cloze task, where the model predicts missing or incorrect code tokens based on surrounding context. AlphaRepair, introduced in this context, leverages large pre-trained code models for APR without any fine-tuning on historical bug fixes, treating repair as a fill-in-the-blank problem rather than a translation task. By doing so, it substantially outperforms state-of-the-art APR techniques on the Defects4J benchmark, demonstrating that learning-based repair is viable even without access to bug-fixing commits [44]. This finding is corroborated by GAMMA, which revisits template-based APR by integrating mask prediction. GAMMA transforms fix templates into mask patterns and utilizes pre-trained models like UniXcoder to predict correct code tokens directly, achieving significant improvements over previous template-based and learning-based methods by bypassing the need for explicit donor code retrieval [3].

Further empirical evidence supports the potency of zero-shot and prompt-based strategies across various model architectures and repair settings. An extensive study applying nine state-of-the-art pre-trained language models (PLMs) to APR in zero-shot settings reveals that directly applying these models, particularly larger ones, can outperform existing APR techniques across multiple datasets and languages [6]. Crucially, this research identifies that infilling-style prompts, which utilize both prefix and suffix code context, are vital for generating compilable and correct patches, highlighting the importance of bidirectional context in repair tasks [6]. This aligns with findings from Round-Trip Translation (RTT), a unique zero-shot strategy that translates buggy code into an intermediate language (or natural language) and back. RTT leverages the model's tendency to regress toward the mean of "natural," bug-free code patterns found in its training corpus, successfully generating plausible patches for bugs missed by fine-tuned models and suggesting that latent correction capabilities exist within general-purpose LLMs [Assessing the Latent Automated Program Repair Capabilities of Large Language Models using Round-Trip Translation].

The effectiveness of zero-shot approaches is further enhanced by sophisticated prompt engineering techniques that incorporate domain knowledge and execution feedback. FitRepair combines direct LLM usage with prompting strategies that retrieve project-specific code ingredients, effectively automating the plastic surgery hypothesis without intensive manual heuristic design [199]. Similarly, ChatRepair introduces a conversation-driven approach that interleaves patch generation with instant feedback from test failures. By constructing prompts that include error messages and previous failed attempts, ChatRepair achieves state-of-the-art results, demonstrating that iterative prompting can mimic the debugging process more effectively than static generate-and-validate paradigms [Keep the Conversation Going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT]. Additionally, augmenting prompts with runtime execution traces has been investigated; while simply adding traces yields limited improvements, LLM-optimized prompts incorporating trace information can outperform trace-free baselines and even small fine-tuned models, indicating that the design of the prompt is as critical as the information it contains [97].

Despite the promise of zero-shot methods, the literature presents nuanced comparisons regarding the trade-offs between prompt engineering and fine-tuning. While some studies note that zero-shot Code Language Models (CLMs) can fix significantly more bugs than traditional deep learning techniques, others argue that fine-tuning can provide substantial gains, ranging from 31% to over 1,000% improvement depending on the dataset [73]. However, conflicting evidence suggests that full fine-tuning can sometimes degrade performance due to overfitting or data distribution shifts, whereas parameter-efficient fine-tuning (PEFT) or zero-shot approaches often yield more robust results [132]. This tension is further explored in research arguing that in data-scarce scenarios, prompt tuning with domain knowledge outperforms standard fine-tuning by an average of 87.33% [226]. Moreover, even small language models (SLMs) can achieve competitive zero-shot performance comparable to larger LLMs, especially when quantized, offering a cost-effective alternative for resource-constrained environments [131].

The versatility of zero-shot and prompt-based repair extends to specialized domains such as security vulnerability fixing and performance optimization. Retrieval-Augmented Prompt Generation (RAPGen) demonstrates that retrieval-augmented prompt generation can effectively address performance bugs without fine-tuning, retrieving instructions from a knowledge base to guide the LLM [72]. Similarly, by reducing code context to relevant snippets via program analysis, LLMs can be queried in a few-shot or zero-shot manner to fix complex security vulnerabilities, outperforming window-based models [194]. These studies collectively indicate that the bottleneck in APR is shifting from data availability for fine-tuning to the strategic design of prompts and the efficient retrieval of contextual information. The consensus across these works is that pre-trained models, when guided by well-constructed prompts that leverage context, execution feedback, and domain knowledge, offer a powerful and efficient pathway for automated program repair that often rivals or exceeds the capabilities of heavily fine-tuned counterparts.

Non-Autoregressive and Efficient Generation

The deployment of Automated Program Repair (APR) tools in latency-sensitive environments, such as continuous integration (CI) pipelines and interactive development environments, is significantly hindered by the inherent delays of autoregressive (AR) generation models. Traditional deep learning-based APR techniques predominantly rely on sequence-to-sequence architectures that generate target code token-by-token, predicting future values based on past outputs. While effective for accuracy, this sequential dependency creates a substantial bottleneck, particularly when repairing complex bugs that require modifications to long code sequences. The resulting time delays often exceed the patience of developers and the strict time budgets of CI systems, limiting the practical adoption of machine learning-based repair tools [188]. To address this critical efficiency challenge, recent research has pivoted toward Non-Autoregressive (NAR) models, which generate target code in parallel, offering the potential for drastic reductions in inference time without compromising repair quality.

However, the direct adaptation of NAR models from machine translation to program repair is non-trivial due to the unique structural and semantic constraints of source code. Naive applications of NAR techniques to APR often suffer from compromised patch quality, manifesting through three primary limitations: over-correction, lack of inter-token dependency information, and insufficient contextual awareness. The over-correction issue arises because NAR models in machine translation typically modify every token in the source sequence, whereas program repair often requires modifying only a small fraction of the code while preserving the majority of the original logic. Without specific mechanisms to identify unchanged tokens, NAR models may erroneously alter correct code segments, introducing new defects [10]. Furthermore, the parallel generation nature of NAR models inherently disrupts the sequential dependency modeling that AR models leverage naturally. In programming languages, where formal grammar and strict structural dependencies are paramount, the loss of inter-token dependency information can lead to syntactically invalid or semantically incorrect patches [188]. Finally, the inability to effectively utilize contextual information during parallel decoding further degrades the model's ability to synthesize accurate fixes.

To overcome these specific challenges, specialized architectures like NARRepair have been proposed as the first customized NAR code generation models for APR. NARRepair introduces three novel components designed to mitigate the limitations of naive NAR approaches. First, to address over-correction, it employs an action-guided repair mechanism. Instead of directly predicting the correct token from a vast vocabulary, the model predicts a repair action for each token-categorized as "keep," "replace," "insert," or "delete." This action-based prediction significantly reduces the complexity of the probability distribution the model must learn, allowing it to confidently retain correct code segments and focus generative capacity on faulty regions [10]. Second, to restore inter-token dependency information lost during parallel generation, NARRepair integrates an inter-token dependency extractor based on Abstract Syntax Trees (AST). By leveraging the nearest common parent nodes in the AST to represent dependencies between token pairs, the model can inject structural and semantic relationship information into the parallel decoding process, ensuring that generated patches adhere to the grammatical rules of the programming language [188]. Third, to alleviate the lack of contextual information, NARRepair utilizes a two-stage decoder. In the first stage, the model generates a preliminary set of fixed tokens, retaining high-confidence predictions while masking low-confidence ones. The second stage then regenerates the masked tokens using the high-confidence tokens from the first stage as rich contextual input, effectively simulating a form of iterative refinement within a non-autoregressive framework [10].

Empirical evaluations demonstrate that these architectural innovations yield substantial performance gains. NARRepair has been shown to achieve repair speeds 1.4 to 6.4 times faster than AR-based APR techniques in GPU environments, with even more pronounced speedups observed in CPU environments, ranging from 5.4 to 18.6 times faster depending on the specific configuration [188] [10]. Crucially, these efficiency improvements do not come at the expense of accuracy; NARRepair maintains state-of-the-art comprehensive performance, fixing more bugs than baseline models within strict time limits across datasets such as Defects4J and QuixBugs. This balance of speed and accuracy highlights the viability of NAR models for practical APR deployment.

While NAR models address the fundamental latency of generation, other efficiency strategies focus on optimizing the search and refinement processes of Large Language Model (LLM)-based APR. For instance, approaches like FLAMES introduce semantic-guided search to replace traditional beam search, which is often memory-intensive and prone to out-of-memory crashes when scaling beam sizes. By utilizing greedy decoding combined with semantic feedback from

test validations, FLAMES significantly reduces VRAM consumption by up to 83% while accelerating the repair process and improving fix rates [240]. Similarly, parameter-efficient fine-tuning (PEFT) techniques have been explored to adapt large pre-trained models for APR with reduced computational costs, demonstrating that methods like $(IA)^3$ can enhance model creativity and fixing capabilities without the resource overhead of full-model fine-tuning [277]. Additionally, cost-aware frameworks like CIGAR and WILLIAMT optimize prompt engineering and patch generation strategies to minimize token usage, achieving significant cost reductions while maintaining high bug-fixing rates [164] [Fixing 7,400 Bugs for 1\$: Cheap Crash-Site Program Repair]. These diverse efficiency strategies, ranging from architectural shifts to NAR generation to optimized search and fine-tuning protocols, collectively advance the feasibility of integrating automated repair tools into modern, high-velocity software development workflows.

Advanced Reasoning, Agentic Frameworks, and Iterative Refinement

The transition from single-pass code generation to iterative, agentic frameworks represents a fundamental paradigm shift in addressing the "near-miss syndrome," a phenomenon where large language models generate code that is semantically proximate to a correct solution but fails functional verification due to minor logical or syntactic errors. Early approaches relied on generating multiple candidates and filtering them based on pass/fail signals, but recent research demonstrates that treating program repair as a dynamic search problem yields superior results. This evolution moves beyond static translation tasks to emulate human debugging behaviors through sophisticated feedback loops, self-correction mechanisms, and collaborative workflows. The foundational architecture for this shift is the synthesize-execute-debug cycle, which has been formalized into multi-agent systems like SEIDR. This framework orchestrates distinct agents for synthesis, execution, instruction, debugging, and ranking, effectively balancing the trade-off between repairing failed programs and generating entirely new ones. By employing selection algorithms such as lexicase and tournament selection, SEIDR addresses the last-mile problem in program synthesis, outperforming conventional genetic programming and standalone large language model usage on benchmarks like PSB2 and HumanEval-X [109]. Empirical studies confirm that base models benefit significantly more from creating patches in an iterative fashion rather than generating multiple outputs simultaneously, with the advantages of iteration becoming even more pronounced in complex benchmarks [241].

A critical evolution in these frameworks is the move from coarse-grained pass/fail signals to fine-grained dynamic state analysis. Traditional automated program repair often relies on static analysis or simple exception messages, which fail to capture the intermediate runtime states necessary for root cause analysis. To bridge this gap, frameworks like VulDebugger and DynaFix introduce dynamic state-guided repair. VulDebugger mimics human developers by setting breakpoints to inspect actual program states and comparing them against expected states inferred from crash-free constraints, significantly improving vulnerability repair rates on real-life projects [147]. Similarly, DynaFix leverages execution-level dynamic information, such as variable states, control-flow paths, and call stacks, collected via instrumentation tools. By iteratively refining patches based on this rich runtime feedback, DynaFix achieves substantial improvements in repair rates over baselines that rely solely on static information, effectively reducing the patch search space [119]. This aligns with findings from DebugRepair, which demonstrates that equipping large language models with intermediate runtime evidence through simulated instrumentation allows for more precise patch refinement than relying on outcome-level failure symptoms alone [32]. Furthermore, TraceCoder enhances this process by capturing fine-grained runtime traces via diagnostic probes and employing a Historical Lesson Learning Mechanism to prevent the recurrence of similar mistakes, achieving significant relative improvements in accuracy [202].

The complexity of modern software systems necessitates collaborative agent architectures capable of managing context across multiple files and domains. Single-agent approaches often struggle with context window limitations and the semantic dispersion of faults in large codebases. RepoRepair addresses this by leveraging large language models to generate hierarchical code documentation, creating structured semantic abstractions that enable precise fault localization and repair across entire repositories, achieving state-of-the-art performance on SWE-bench Multimodal [27]. In specialized domains, the adaptability of agentic frameworks is equally evident. For hardware design, the Clover and R3A frameworks employ stochastic tree-of-thoughts and multi-agent fault localization to navigate the vast search space of RTL repairs, balancing exploration and exploitation to achieve high reliability where traditional template-based methods fail [222] [278]. Similarly, the development of lightweight, language-specific frameworks like RAMP for Ruby highlights the potential of tailored multi-agent systems to achieve high repair rates without relying on costly fine-tuning, proving that efficient, feedback-driven loops can be effective even for under-studied languages [217]. In educational contexts, frameworks like RePurr adapt these principles to block-based programming, overcoming the lack of redundancy in learner code by exploiting model solutions as fix sources [RePurr: Automated Repair of Block-Based Learners' Programs].

Advanced reasoning strategies also incorporate search algorithms and specification-centric paradigms to enhance repair reliability and reduce hallucinations. Monte Carlo Tree Search has been successfully integrated into frameworks like APRMCTS and CodePilot to perform global evaluations of explored patches, preventing the search from falling into local optima and improving efficiency compared to trial-and-error strategies [34] [232]. Alternatively, VibeRepair proposes a specification-centric approach that shifts the focus from direct code editing to repairing behavior

specifications. By translating buggy code into structured behavior specifications, inferring intended behaviors, and synthesizing code from corrected specifications, this method reduces the patch search space and mitigates hallucinations, demonstrating superior effectiveness on Defects4J benchmarks [90]. Additionally, the concept of "cogeneration," where agents generate both fixes and bug reproduction tests simultaneously, has been shown to increase developer confidence and reduce engineering effort without compromising fix generation rates [116] [38].

Despite these advancements, challenges remain regarding the cost-effectiveness and generalizability of agentic systems. While iterative and multi-agent approaches significantly boost success rates, they can incur high computational and token costs. Research indicates that while larger models generally perform better, fine-tuned smaller models can sometimes outperform general-purpose large models, suggesting that model specialization and efficient prompt engineering are as critical as model scale [60]. Furthermore, the efficacy of self-repair is sometimes bottlenecked by the model's ability to provide accurate feedback on its own code; using stronger models for feedback or incorporating human-level debugging signals can yield substantially larger performance gains [101]. The integration of diverse software artifacts-such as issue descriptions, stack traces, and debugging information-is also vital, as demonstrated by DEVLore, which shows that combining these artifacts allows large language models to mimic human developers more closely than methods relying on single types of software information [21]. Ultimately, the convergence of dynamic execution feedback, collaborative agent roles, and structured search strategies represents a paradigm shift toward fully autonomous programming systems capable of handling complex, real-world debugging scenarios.

Iterative Debugging and Self-Correction Mechanisms

The paradigm of Automated Program Repair (APR) has undergone a fundamental transformation, shifting from static, generate-and-validate models to dynamic, agentic frameworks that emulate the iterative reasoning processes of human developers. This evolution is primarily driven by the recognition of the "near-miss syndrome," where Large Language Models (LLMs) generate code that semantically resembles a correct solution but fails unit tests due to minor logical or syntactic imperfections [109] [104]. Addressing these near-misses necessitates moving beyond single-pass generation toward multi-turn dialogues and sequential decision-making processes that leverage immediate feedback from compilers, test suites, and runtime environments. Conversational agents have emerged as the primary vehicle for implementing these iterative loops, transforming repair into a dialogue where the model learns from its own failures. Systems like ChatRepair interleave patch generation with instant feedback, allowing the agent to incorporate test failure information directly into subsequent prompts to avoid repeating mistakes, a strategy that significantly outperforms traditional methods that discard failed patches [Keep the Conversation Going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT] [228].

However, the efficacy of such conversational repair is heavily contingent on the quality and structure of the feedback provided. Research indicates that structured feedback, particularly in the form of test results, yields higher success rates than unstructured natural language hints, although the marginal benefits of iteration tend to diminish after two or three rounds if the feedback lacks specificity [248]. To enhance the informativeness of this feedback, ContrastRepair introduces contrastive test case pairs-consisting of both failing and passing tests-to help the LLM isolate root causes by minimizing the difference between the generated passing test and the given failing test [201]. Similarly, in educational contexts, frameworks like Cref leverage tutor guidance within a conversational loop to significantly improve repair capabilities, demonstrating that external expert signals can overcome the limitations of self-correction [276]. Despite these advancements, studies suggest that self-repair capabilities are often bottlenecked by the model's ability to provide accurate feedback on its own code, with performance gains varying significantly across different problem subsets and models [101]. Furthermore, the stability of LLM outputs can be compromised by temperature settings, leading to functional failures in high-temperature iterative scenarios, indicating that naive self-correction is not a universal solution [120].

To overcome the limitations of coarse-grained pass/fail signals, advanced agentic frameworks employ sophisticated search strategies and dynamic execution data to navigate the vast patch space more effectively. DynaFix argues that relying solely on static analysis or simple test outcomes limits repair effectiveness; instead, it iteratively captures detailed execution traces, including variable states and call stacks, to guide the LLM in a stepwise debugging process that mirrors human practices [119]. Complementing this, DebugRepair enhances patch refinement by simulating the human debugging workflow: it purifies test semantics, instruments code to collect intermediate runtime states, and uses this evidence to drive a hierarchical conversational repair process [32]. TraceCoder further emulates the human observe-analyze-repair cycle by utilizing a Historical Lesson Learning Mechanism to distill insights from prior failures, preventing the recurrence of similar mistakes and enforcing strict improvement through a rollback mechanism [202]. For security vulnerabilities, where static analysis often fails to capture complex state interactions, VulDebugger acts as a debugging agent that inspects actual program states via breakpoints and compares them against expected states derived from crash-free constraints [147].

The architectural design of these iterative systems often involves multi-agent collaboration to specialize tasks and improve robustness. The SEIDR framework exemplifies this by utilizing a multi-agent architecture that synthesizes, executes, instructs, debugs, and ranks candidate solutions, employing evolutionary strategies like tournament and lexibase selection to retain promising programs across generations [109]. Specialized agents have also been developed to address specific bottlenecks; for instance, RAMP employs a team of collaborative agents specializing in test design, execution, and feedback integration to achieve high repair rates in under-studied languages like Ruby without costly fine-tuning [217]. In the hardware domain, Clover and R3A utilize stochastic tree-of-thoughts and multi-agent fault localization to manage the randomness of LLMs and ensure reliable RTL repair [222] [278]. Furthermore, the cogeneration of Bug Reproduction Tests (BRTs) alongside fixes, as seen in industrial deployments, significantly increases confidence in generated patches and improves overall resolution rates [116] [38]. To mitigate error propagation and stabilize the repair loop, recent works like TokenRepair introduce internal reflection mechanisms that

analyze token-level uncertainty to identify and refine suspicious code segments before external validation, complementing the external feedback loop with fine-grained internal diagnostics [280]. Additionally, EVIACT coordinates evidence-driven guardrails across retrieval, compilation, and testing stages to ensure that repair actions are grounded in concrete execution evidence, thereby reducing API costs and improving resolve rates [141]. These developments collectively highlight a trajectory toward more robust, evidence-grounded, and cognitively aligned iterative repair systems that move beyond simple trial-and-error toward genuine automated debugging.

Multi-Agent Collaboration and Role Specialization

The evolution of Automated Program Repair (APR) has marked a decisive shift from monolithic, single-agent approaches toward sophisticated multi-agent frameworks that decompose the repair process into distinct, specialized roles. This architectural transition is primarily driven by the need to overcome the "near-miss syndrome," a phenomenon where Large Language Models (LLMs) generate code that semantically resembles a correct solution but fails unit tests due to minor, hard-to-detect errors [109]. By emulating the collaborative workflows of human development teams, these systems assign specific responsibilities-such as fault localization, test generation, code synthesis, and validation-to dedicated agents. This division of labor creates a robust feedback loop that iteratively refines candidate solutions, enabling deeper semantic reasoning and more effective error handling than standalone models constrained by context limits or reasoning bottlenecks [109].

A primary advantage of multi-agent systems is the decomposition of the "observe-analyze-repair" cycle into granular steps, allowing agents to focus on specific aspects of the debugging process while leveraging execution-level dynamic information. Traditional static analysis often misses critical runtime behaviors, leading to imprecise fault localization. To address this, frameworks like DynaFix iteratively leverage runtime information, such as variable states and call stacks, transforming them into structured prompts to guide LLMs in generating candidate patches [119]. Building on this, DebugRepair enhances patch refinement by enabling agents to simulate the insertion of instrumentation to collect intermediate runtime states, providing richer feedback than simple exception messages and stabilizing the iterative repair loop [32]. Similarly, TraceCoder employs a collaborative framework where agents instrument code with diagnostic probes to capture fine-grained runtime traces, conduct causal analysis to identify root causes, and utilize a Historical Lesson Learning Mechanism to prevent the recurrence of similar mistakes [202]. In the domain of security, VulDebugger acts as an agent that mimics human developers by setting breakpoints and comparing actual program states against expected states derived from crash-free constraints, a capability that static-only agents lack [147]. These systems collectively illustrate that the integration of dynamic, execution-level feedback is essential for resolving complex bugs that require multi-step reasoning.

Role specialization is particularly critical in addressing the challenges of fault localization within large codebases, where traditional methods struggle to pinpoint exact error locations amidst millions of tokens. To mitigate this, frameworks like RGFL introduce a hierarchical reasoning module where agents generate structured, bug-specific explanations for candidate files and elements, significantly improving localization accuracy over baseline methods that rely on surface-level similarity [70]. Complementing this, RepoRepair leverages a documentation-enhanced approach where agents generate hierarchical code documentation to create structured semantic abstractions, enabling the system to comprehend repository-level context and dependencies that are otherwise inaccessible to standard retrieval mechanisms [27]. The effectiveness of these specialized roles is also evident in domain-specific applications; in Ruby, the RAMP framework employs a team of collaborative agents including a Programmer, Test Designer, and Feedback Integrator to achieve high repair rates without relying on large multilingual databases, demonstrating that lightweight, role-specific prompting can be highly effective [217]. In hardware design, Clover and R3A demonstrate the efficacy of dispatching tasks to specialized agents or symbolic solvers based on the nature of the repair operation, utilizing stochastic Tree-of-Thoughts mechanisms to balance exploration and exploitation for reliable outcomes in Register Transfer Level (RTL) repair [222; 278].

The interaction between agents often involves a rigorous cycle of test generation and validation, which serves as a guardrail against hallucinated or overfitted patches. Research indicates that cogenerating Bug Reproduction Tests (BRTs) alongside fixes allows agents to validate their own hypotheses more effectively, reducing the engineering effort required to maintain separate generation pipelines [116; 38]. Systems like EVIACT coordinate three evidence-driven guardrails-a retrieval scaffold, a compile gate, and a test-driven gate-to ensure that patches are not only syntactically valid but also recover target tests before undergoing full regression [141]. To further reduce the risk of overfitting to a single interpretation of the bug, AdverIntent-Agent utilizes a reasoning agent to infer multiple adversarial program intents, a test agent to generate corresponding test cases, and a repair agent to synthesize patches, thereby exploring diverse program intents [76]. Conversely, approaches like TokenRepair integrate internal reflection mechanisms where agents analyze token-level uncertainty to identify suspicious code segments before applying external feedback, preventing error propagation in the iterative loop [280]. These diverse implementations underscore that while the

overarching goal of automating repair remains constant, the optimal configuration of agent roles and their interaction protocols depends heavily on the specific constraints of the programming language, the complexity of the bug, and the available execution environment.

Tree Search and Global Exploration Strategies

The integration of Large Language Models (LLMs) into Automated Program Repair (APR) has catalyzed a paradigm shift from rigid, template-based fixes to dynamic, generative approaches. However, early LLM-based methods often relied on simple trial-and-error strategies, generating patches through independent sampling without leveraging information from previous attempts. This approach suffers from inherent limitations, including redundant exploration and a tendency to become trapped in local optima, where the model repeatedly generates similar incorrect patches [85]. To address these inefficiencies, recent research has increasingly adopted tree search algorithms, such as Monte Carlo Tree Search (MCTS) and best-first search, to enable global evaluation of the patch space and guide the reasoning process of agentic frameworks. These strategies transform the repair process from a myopic, step-by-step generation task into a structured planning problem, allowing systems to navigate vast search spaces more intelligently.

A primary motivation for incorporating tree search is the need to overcome the short-sighted nature of standard autoregressive generation. Traditional generate-and-validate paradigms often fail to plan several steps ahead, leading to suboptimal repair trajectories. [34] explicitly addresses this by integrating MCTS into the APR workflow. Unlike methods that sample patches independently, APRMCTS performs a global evaluation of explored patches, selecting the most promising nodes for subsequent refinement and generation. This iterative tree search allows the system to avoid local optima and significantly improves search efficiency. Empirical results demonstrate that this strategy enables models like GPT-3.5 to fix substantially more bugs while utilizing far fewer patches compared to baseline methods that rely on massive sampling sizes. Similarly, [232] introduces CodePilot, a hybrid framework that combines LLMs with MCTS for repository-level issue resolution. By using execution feedback as a reward signal within the MCTS process, CodePilot explores diverse patch trajectories and decomposes complex debugging tasks into structured reasoning steps, achieving superior resolve rates on benchmarks like SWE-bench Lite.

Beyond MCTS, other tree-based structures have been proposed to enhance the precision of candidate solution generation. [36] argues that while Chain-of-Thought (CoT) techniques improve reasoning, their application in APR remains insufficient for complex, multi-step logic. The T3 framework integrates LLM reasoning capabilities with a multi-level tree search, effectively guiding the selection of repair strategies and optimizing sample selection. This approach establishes a robust framework for achieving efficient automated debugging by systematically exploring the solution space rather than relying on stochastic generation alone. In the domain of hardware description languages, [278] adapts these concepts through a stochastic Tree-of-Thoughts method. This technique controls a patch generation agent to explore validated solutions, balancing exploration and exploitation via a heuristic function to ensure reliable outcomes in RTL repair, a domain characterized by long input contexts and high stakes for correctness.

The efficacy of tree search is further amplified when combined with semantic feedback and execution guidance. [240] introduces FLAMES, which replaces memory-intensive beam search with a semantic-guided best-first search algorithm. FLAMES utilizes greedy decoding to maintain memory efficiency while steering the search toward promising candidates using semantic feedback from test validations, such as the number of passing and failing test cases. This method not only reduces VRAM consumption significantly but also accelerates the repair process, outperforming state-of-the-art techniques that rely on conventional beam search. Furthermore, the concept of global exploration is evident in agentic frameworks that interleave generation with instant feedback. [85] proposes ChatRepair, which, while conversational in nature, embodies the principles of tree search by learning from both failures and successes of earlier attempts. By constructing new prompts based on incorrect patches and their corresponding test failures, ChatRepair avoids repeating mistakes and builds upon plausible patches, effectively traversing the search tree in a directed manner.

Advanced reasoning agents also benefit from structured exploration strategies that mimic human debugging processes. [32] enhances patch refinement by incorporating intermediate runtime evidence collected through simulated debugging. While not explicitly labeled as a tree search, its hierarchical, iterative process of refining candidate patches based on newly observed runtime states parallels the node expansion and backtracking mechanisms found in tree search algorithms. Similarly, [141] coordinates evidence-driven guardrails across repair stages, utilizing retrieval scaffolds and test-driven gates to filter invalid edits. This structured approach ensures that the agent's exploration of the repair space is grounded in execution evidence, reducing the likelihood of diverging into unproductive search paths.

Despite the clear advantages, challenges remain in balancing the computational cost of tree search with repair effectiveness. [241] investigates the trade-off between generating multiple outputs in a single round versus multiple rounds of iteration. The study finds that iterative strategies, which align closely with the step-wise expansion of tree search, provide significant benefits, particularly for complex benchmarks, though diminishing returns can occur if thresholds are exceeded. Moreover, [280] highlights that relying solely on coarse-grained external feedback (pass/fail) can lead to inefficient refinement. By integrating internal reflection to localize faulty tokens before expanding the search, frameworks can achieve more targeted and fine-grained corrections, suggesting that future tree search implementations should incorporate finer-grained heuristics to guide node selection. The synthesis of these works indicates a consensus that global exploration strategies are critical for advancing LLM-based APR, enabling systems to navigate the vast search space of potential patches more intelligently and ensuring that computational resources are allocated to the most promising repair trajectories.

Context Awareness, Fault Localization, and Retrieval Mechanisms

Accurate fault localization and the strategic retrieval of relevant context constitute the foundational prerequisites for successful Large Language Model (LLM)-based Automated Program Repair (APR), particularly within large-scale repositories where defects frequently span multiple files and modules. The efficacy of modern repair pipelines is increasingly dependent on how effectively systems can narrow the search space while simultaneously providing sufficient semantic depth to understand the root cause of a defect. Recent empirical studies challenge the traditional assumption that increasingly precise localization uniformly leads to better repair outcomes, revealing a nuanced trade-off where broader semantic understanding at higher abstraction levels often complements precise line-level identification. For instance, [210] demonstrates through a large-scale study that file-level localization is the dominant factor, yielding a 15-17x improvement over baselines with no file context. However, the study reveals that expanding context beyond the buggy files does not always correlate with performance gains; specifically, line-level context expansion frequently degrades performance due to noise amplification, whereas element-level expansion offers conditional benefits dependent on the quality of the file context. This finding contradicts the intuition that more granular context is inherently superior, suggesting that LLMs benefit more from a broad semantic view of the repository than from excessive surrounding code lines that may dilute the repair signal. Supporting this perspective, [230] provides empirical evidence that function-level granularity often yields the highest repair rates in repository-scale scenarios, challenging the push for ever-finer granularity in complex, multi-file environments.

Despite the efficacy of coarser granularities for context provision, other research argues that fine-grained targeting remains essential for the actual editing phase to prevent patch overfitting and reduce the search space. [Toggle: Token-Granulated Bug Localization and Repair] argues that token-level localization injects strong inductive bias by preventing the regeneration of non-buggy code prefixes and suffixes, though it requires careful handling of tokenizer inconsistencies between localization and repair models. Similarly, [229] utilizes Abstract Syntax Tree (AST) paths to predict specific buggy tokens and change operators, demonstrating that reducing the search space to exact tokens can eliminate patch overfitting and improve efficiency. These divergent findings suggest that the optimal strategy is not a single fixed granularity but a hybrid approach: using broader file or function-level context for semantic understanding and root cause analysis, while employing token or line-level precision to constrain the generation of the actual patch.

To address the limitations of shallow retrieval and structural heuristics, recent methodologies have integrated diverse software artifacts and advanced reasoning mechanisms to enhance context awareness. Human developers typically synthesize information from issue descriptions, error stack traces, and debugging sessions to diagnose faults, a practice that many early LLM-based approaches failed to emulate. [21] proposes the DEVLoRe framework, which mimics this human workflow by combining issue content, stack traces, and debugging information. Their results indicate that while issue content is most effective for method localization, stack traces are superior for guiding the actual repair, highlighting the complementary nature of different artifact types. Furthermore, the integration of historical data has proven valuable; [65] leverages historical heuristics from blame commits to augment LLM prompts, showing that past development context significantly enhances bug-fixing performance compared to using only the current code snapshot.

The challenge of repository-scale repair has also spurred innovations in retrieval-augmented generation (RAG) and knowledge representation to bridge the gap between high-level issue descriptions and low-level code changes. Standard retrieval methods often struggle with the sheer volume of tokens in large codebases, leading to context window overflow or the inclusion of irrelevant data. [27] addresses this by generating hierarchical code documentation to create structured semantic abstractions, enabling LLMs to comprehend repository-level dependencies without processing raw code for every file. This approach achieves state-of-the-art performance on complex benchmarks by filtering suspicious files and functions through these abstractions. Complementing this, [185] proposes KGCompass, which utilizes a repository-aware knowledge graph to link issues with code entities. This method allows for multi-hop traversals to identify relevant context, proving that structural knowledge graphs can effectively narrow the search space to a manageable set of candidate functions, a task where purely embedding-based retrieval often fails. Additionally, [70] introduces a hierarchical reasoning module that generates structured, bug-specific explanations for candidate files and elements before ranking them. By shifting from simple textual matching to explicit causal reasoning, RGFL significantly improves localization accuracy, demonstrating that LLMs can better capture semantic links between bug reports and code when prompted to explain why a location is suspicious rather than just identifying similarity.

The integration of retrieval mechanisms extends beyond static code analysis to include dynamic execution feedback and internal model states. [32] introduces a self-directed debugging framework where the LLM autonomously inserts instrumentation to collect intermediate runtime states. This approach moves beyond outcome-level failure symptoms, such as stack traces, to provide the model with rich, dynamic context about the program's execution flow, thereby facilitating more accurate root-cause analysis. Furthermore, [280] proposes utilizing internal reflection by analyzing context-aware token-level uncertainty fluctuations to identify suspicious tokens within a generated patch. This internal signal serves as a fine-grained context source that guides targeted refinement, complementing external test feedback. These diverse strategies collectively indicate that the future of LLM-based APR lies not in a single optimal localization granularity, but in adaptive systems that dynamically select and retrieve context based on the specific characteristics of the bug, the available artifacts, and the semantic depth required for a successful fix.

Granularity of Fault Localization

The evolution of automated program repair has witnessed a decisive paradigm shift from coarse-grained fault localization, which typically identifies buggy methods or lines, to fine-grained, token-level localization. This transition is driven by the critical necessity to constrain the search space for patch generation and to prevent the regeneration of non-buggy code segments, a primary source of inefficiency and overfitting in generate-and-validate approaches. Traditional spectrum-based fault localization often yields results at the statement level, forcing repair tools to explore a vast number of change trials on non-faulty elements within a suspicious line. To address this, recent frameworks have adopted specialized architectures and large language models to pinpoint specific buggy tokens, thereby enabling more precise and efficient repairs. A pioneering approach in this domain is Beep, which utilizes abstract syntax tree paths to predict not only the specific buggy code element but also the requisite change operator, such as delete, update, or insert. By training on massive datasets, [229] demonstrates that identifying buggy tokens significantly reduces debugging effort compared to line-level localization. The study highlights that coarse granularity leads automated program repair tools to generate plausible but incorrect patches because they modify non-buggy elements within a localized statement. In contrast, the token-level precision of Beep allows repair pipelines to achieve a 100% correctness ratio in generated patches by focusing exclusively on the identified faulty tokens, thus eliminating the need to regenerate shared prefixes and suffixes.

Building upon the concept of token granularity, [14] introduces Toggle, a framework that explicitly separates bug localization and fixing using distinct large language models. Toggle employs an encoder model to identify the start and end tokens of a bug and a decoder model to generate the fix. A critical insight from this work is that token-granular localization injects a strong inductive bias into the repair process, preventing the model from hallucinating changes in correct code segments. However, [14] also identifies a significant challenge: tokenizer inconsistencies between the localization and fixing models can misalign the predicted bug location with the generation context. To mitigate this, Toggle incorporates an adjustment module to bridge the discrepancy, ensuring that the fine-grained localization signal effectively guides the repair generation. The benefits of fine-grained localization extend beyond static prediction to iterative refinement processes. [280] proposes TokenRepair, which integrates internal reflection to localize faulty tokens within a generated patch based on uncertainty fluctuations. Unlike methods that rely solely on external test feedback, TokenRepair uses token-level suspiciousness scores to guide Chain-of-Thought rewriting, refining only the uncertain tokens rather than regenerating the entire patch. This approach stabilizes the iterative repair loop and mitigates error propagation, demonstrating that internal, fine-grained signals are complementary to external validation. Similarly, [196] utilizes a lightweight diagnostic model to predict line-level suspiciousness signals, which are then used to search over top-k suspicious regions, further validating that fine-grained feedback outperforms coarse-grained self-critiques.

Despite the clear advantages of token-level precision, the optimal granularity for large language model-based repair remains a subject of empirical investigation, particularly in repository-scale contexts. [230] challenges the assumption that finer is always better, finding that function-level granularity sometimes yields higher repair rates than line-level granularity when using ground-truth localization in large codebases. This suggests that while token-level precision minimizes edit distance, it may occasionally omit necessary contextual information required for the large language model to understand the bug's semantic root cause. Corroborating this nuance, [210] finds that while file-level context expansion consistently improves performance, line-level context expansion often introduces noise that degrades repair success. The study concludes that the most effective strategy combines broad semantic context at higher abstraction levels with precise line-level localization for the actual edit, rather than expanding the context window around the buggy line. Furthermore, the integration of fine-grained localization with other software artifacts enhances its efficacy. [21] demonstrates that combining issue content, error stack traces, and debugging information allows large language models to localize bugs more accurately than relying on code structure alone. When these artifacts are used to guide token-level repairs, the system can address complex, non-single-method bugs that coarse-grained approaches miss. Additionally, [191] introduces AdaPatcher, which utilizes a bug locator with self-debug learning to pinpoint locations before applying a program modifier trained to prioritize minimal changes. This aligns with the findings in [35], which argues that standard fine-tuning lacks explicit signals for token preservation, leading to over-editing. By deriving token-level preservation signals, PAFT ensures that edits are concentrated strictly on faulty regions, reinforcing the principle that precise localization is key to generating minimal, high-quality patches.

The tension between granularity and context is also evident in multi-hunk repair scenarios. [94] and [273] illustrate that fixing bugs spanning multiple locations requires re-executing fault localization iteratively or identifying semantically related sibling locations. These approaches rely on the ability to localize specific tokens across different code segments to construct coherent multi-hunk patches, a task that coarse-grained localization fails to support effectively. Ultimately, the literature converges on the view that moving from line-level to token-granular bug localization is essential for preventing the regeneration of non-buggy code, reducing patch overfitting, and enhancing the overall efficiency of automated repair systems, provided that the localization is balanced with sufficient semantic context to guide the repair logic.

Conclusion

The landscape of Automated Program Repair has undergone a paradigmatic transformation, evolving from rigid, heuristic-driven search spaces to flexible, generative ecosystems powered by Large Language Models. This shift has fundamentally redefined the conceptualization of software maintenance, moving beyond the limitations of the "plastic surgery hypothesis" to enable the synthesis of novel logic and the resolution of complex, multi-hunk bugs that were previously intractable. By leveraging vast pre-training corpora, modern approaches have transcended the need for manually crafted templates, facilitating function-level and repository-level repairs through sophisticated retrieval mechanisms and context-aware architectures. The integration of dynamic execution feedback, iterative self-correction, and multi-agent collaboration now mirrors human debugging workflows, providing a robust mechanism to address the "near-miss syndrome" and refine patches through continuous validation loops.

Despite these advancements, significant challenges remain in bridging the gap between benchmark performance and real-world applicability. The field faces critical hurdles regarding trust and robustness, particularly concerning the susceptibility of models to syntactic variations, the risk of hallucination, and the persistence of patch overfitting where generated solutions satisfy test suites without addressing root causes. Current evaluation methodologies are increasingly scrutinized for data leakage and an over-reliance on static test suites that fail to capture semantic correctness or edit minimality. Consequently, the research trajectory is shifting toward more rigorous, leakage-free benchmarks and metrics that prioritize edit precision, naturalness, and cost-efficiency. The development of non-autoregressive architectures and parameter-efficient fine-tuning strategies further underscores a growing emphasis on scalability, ensuring that repair tools can operate within the latency and resource constraints of industrial environments while balancing the trade-offs between generation speed and semantic accuracy.

Looking forward, the future of automated repair lies in the convergence of generative capabilities with formal verification, domain-specific knowledge, and privacy-preserving collaboration. The emergence of specification-centric approaches and neurosymbolic frameworks promises to align generated code more closely with developer intent, reducing semantic errors through mathematical guarantees rather than empirical testing alone. Simultaneously, the adoption of federated learning and synthetic data generation offers pathways to leverage proprietary industrial data without compromising security, addressing the scarcity of high-quality training sets in low-resource and legacy domains. Ultimately, the maturation of this field depends on fostering effective human-AI collaboration, where explainable, context-aware agents act not merely as code generators but as intelligent partners capable of navigating the full complexity of modern software ecosystems, transforming automated repair from an experimental tool into a reliable cornerstone of software engineering.